

A Probabilistic Machine Learning Approach to Shot Quality, Shooting Performance Over Expectation, and Winning in the NBA

Brandon Yoonsu Nam

Proof School, San Francisco, CA

Country/Region: United States

Project Track: Natural & Applied Science (Mathematics)

Abstract

In the NBA, a team's offensive success is based on the quality of shots it generates and the ability of its players to make them. Disentangling these two reasons is a problem of estimating a conditional probability: the chance that the shooter makes a shot given its spatial and contextual features. This is very important and has direct implications for coaching decisions, but current approaches rely on proprietary tracking data, making it inaccessible to most. This paper predicts shot outcomes using only publicly available data, framing it as a supervised binary classification problem. Using multiple seasons of NBA play-by-play and shot-location data, I train machine-learning models to estimate the probability that a shot results in a made basket. These probabilities can be used to calculate expected points: $E[\text{points}] = \text{shot value} \times \text{make probability}$. I then compare expected points with actual points to identify outlier players and teams, compute SPOE (shooting performance over expected), and map these differences to expected wins using the Pythagorean winning percentage formula. My model achieves an AUC of 0.665 and an accuracy of 63.5% using only publicly available data. Two reference points give these numbers scale. First, the league-average field goal percentage in 2024–25 was 46.7%, so a model that predicts a miss every time would be 53.3% accurate. Second, today's proprietary tracking systems publish no accuracy figures to compare against; the closest benchmarks come from the brief window (2013–2016) when the NBA publicly released tracking-derived statistics such as the distance to the closest defender, before discontinuing them. Models trained on that data reached a maximum accuracy of 68% (Oughali et al., 2019). My model, built entirely from features that remain available for every season, covers most of the distance from the 53.3% floor to that tracking-informed result. The results in this paper suggest that shot quality accounts for a substantial portion of winning and can inform roster and coaching decisions without requiring proprietary tracking data.

Introduction and Research Objectives

Shot quality is fundamentally a math problem: given the measurable features of a shot (location, time, score, shot type, etc.), what is the probability that an average shooter makes it? Estimating this probability and using it to separate skill from short-term luck, then linking it to winning, is the central problem tackled by this paper.

In the NBA, teams play 82 regular-season games, and each of the 30 teams takes tens of thousands of shots per season. A 24-second shot clock limits each possession, forcing teams to balance shot selection with time constraints. Small differences in efficiency can amount to several wins or losses, making the quality of shots and player skill at making them central to success.

Figures 1 and 2 represent shot data from the 2024-2025 season to help visualize the scoring environment.

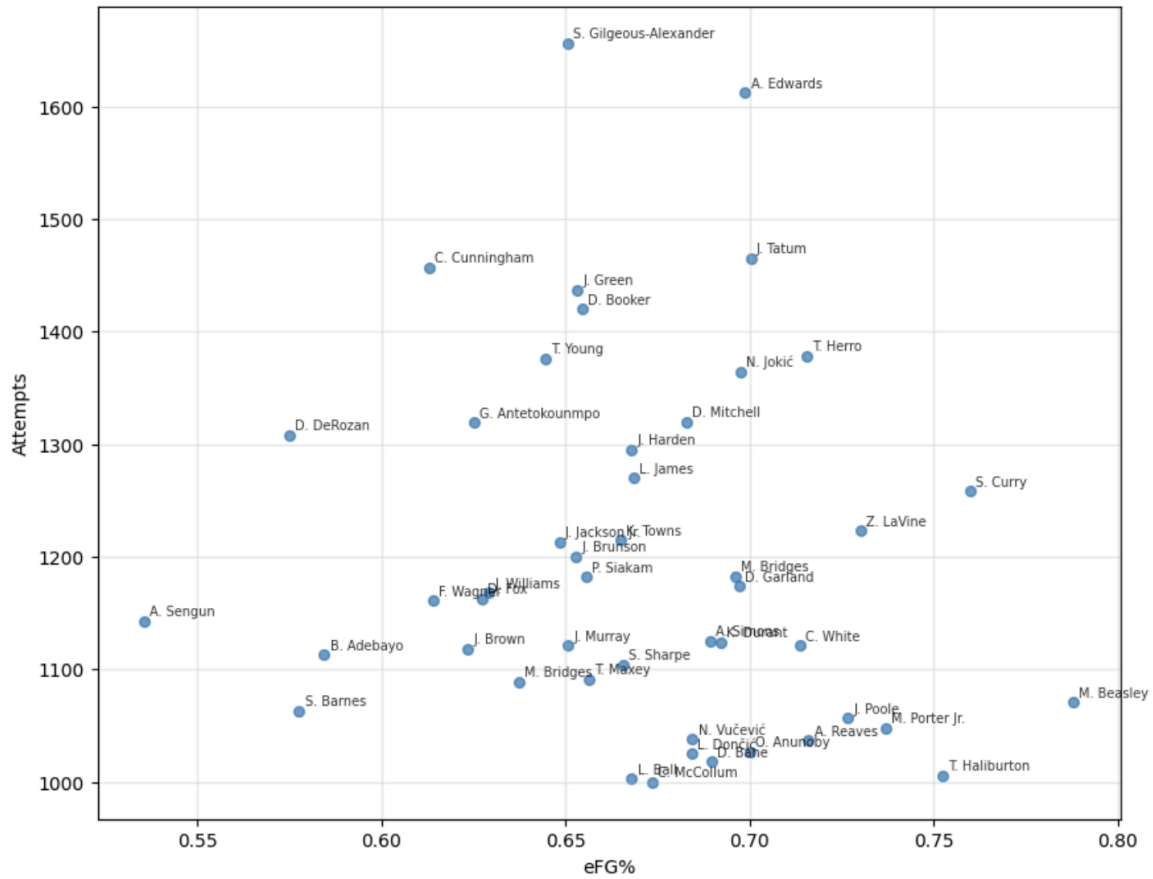


Figure 1. Effective field goal percentage versus shot attempts for NBA players during the 2025 season, restricted to players with at least 1000 attempts.

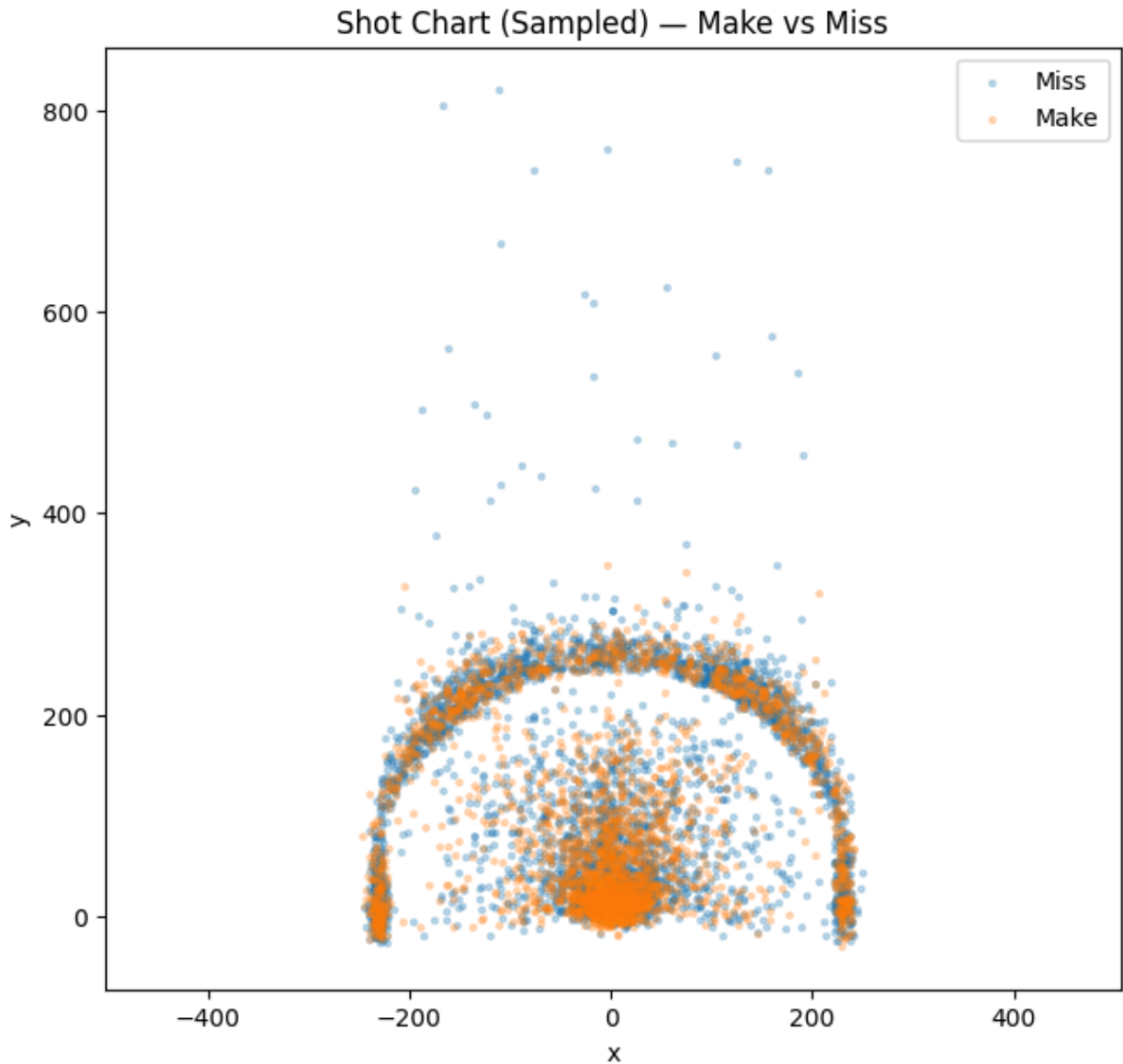


Figure 2. All shot attempts by NBA players from the 2025 season.

Standard statistics, such as field goal percentage and points per game, reveal outcomes but not the quality of shots that led to them. A pull-up mid-range jumper and an open layup seem identical if they both go in, but the process that created each shot is very different. Modern basketball analytics try to separate process from noise and randomness by looking at the physical characteristics of a shot: who took it, from where, under what defensive pressure, and

clock conditions, then calculating how likely it was to go in. This concept is referred to in the literature as shot quality: the probability that a league-average shooter will make a shot, given the physical and contextual characteristics (Cervone et al., 2014). Seminal work presented at the MIT Sloan Sports Analytics Conference by Cervone et al. (2014) modeled make probability by using detailed tracking data, defender distance, and other features. Related research generalized to Expected Possession Value, which assigns a continuous expected point value to a possession (Cervone et al., 2014). Industry solutions such as Second Spectrum's quantified Shot Quality and ShotQuality.com built similar models by using contextual variables to evaluate offense.

Proprietary features like defender distance and ball trajectory are good predictors of shot outcomes (Cervone et al., 2014; Shah & Romijnders, 2016). Shah and Romijnders (2016) achieved an AUC of 0.843 predicting three-point outcomes from ball-tracking data — though their model watches the ball already in flight, a different task from estimating shot quality before the release. For pre-shot prediction, the closest benchmarks come from the brief window (2013–2016) when the NBA publicly released tracking-derived statistics such as closest-defender distance: models built on that data reached a maximum accuracy of 68% before it was discontinued (Oughali et al., 2019). This raises the question of whether features engineered from data that remains publicly available can serve as a substitute. This paper takes the position that carefully engineered features, such as a proxy shot clock and a contest score, can produce a useful shot quality model without exclusive infrastructure.

This paper addresses one main research question: Can a shot prediction model trained on only public data achieve similar performance to proprietary tracking models? Also, two supporting questions: whether outperforming expectations reflects skill or luck, and how much of winning shot quality explains.

Methods

I used publicly available play-by-play data from the Shufinskiy NBA data repository (https://github.com/shufinskiy/nba_data/tree/main/datasets), which pulls from the NBA Stats API v3. All code and data used in this analysis are publicly available at <https://github.com/bnamm1/nba-shot-quality>. The raw data contains approximately 7.05 million events across 12 seasons (2013–14 through 2024–25), with about 575,000 events per season. After filtering down to field goal attempts only, the dataset contains approximately 2.51 million shots (~219,000 per season). Each row corresponds to a single shot attempt, and each column describes some aspect of the shot. The data contains 24 columns per season. These include spatial features (xLegacy, yLegacy, shotDistance), shot outcomes (shotResult, shotValue), game context (clock, period, scoreHome, scoreAway, location), and shot type (actionType, subType). Six additional columns were added during preprocessing, including ABS_TIME, SHOT_CLOCK_APPROX, and contest_score, bringing the total to 30 columns before further feature engineering.

The data was cleaned in several steps. We restrict play-by-play events to field-goal attempts and retain only shots with an unambiguous made/missed outcome so duplicate events, malformed records, and non-regular-season games are filtered out. Shots whose possession chain cannot be reconstructed from the play-by-play, about 2.9% of attempts, are retained with a missing shot-clock value, which is median-imputed at model time.

Unfortunately, publicly available data did not consistently include defender distance or shot clock across multiple seasons. To compensate, additional features were engineered to better reflect what is being represented. First, a proxy shot clock is reconstructed from past play-by-play data by taking the marked time stamps and calculating the remaining time on the clock. Many things

can cause the shot clock to reset. It resets to 24 when possession of the ball changes (e.g., made shots and turnovers). It resets to 14 on stoppages that don't involve a change of possession (e.g., offensive rebounds and defensive fouls) starting in 2018-2019. Figure 3 shows an excerpt of NBA play-by-play data used to reconstruct the shot clock.

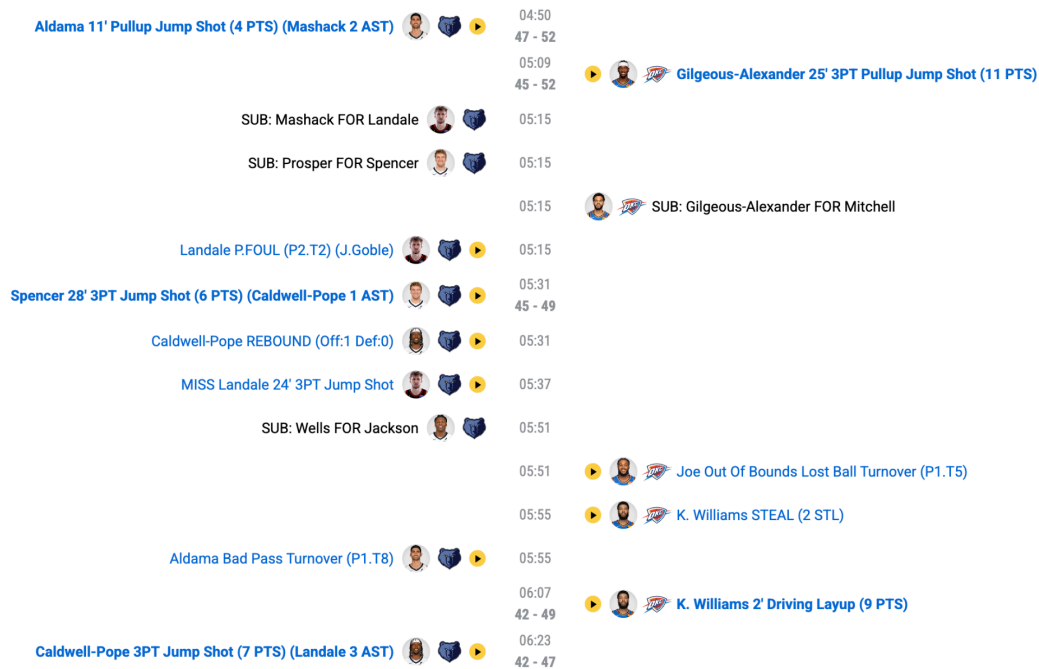


Figure 3. Play-by-play data from the 2nd quarter of Grizzlies vs Thunder on Dec 22, 2025 (from NBA.com).

This procedure yields a reasonable estimate of the shot clock situation for every shot attempt.

The reconstructed clock was validated against the NBA's tracking dashboard, which reports attempts and makes per shot-clock range per player. Since the ground truth is for all the shots of a given player, we use our proxy to count the number of shots in each shot clock bin for each player as well. Then, we compute an upper bound of our accuracy: $\sum_b \min(\text{proxy}_b, \text{truth}_b) / N$ (by the pigeonhole principle). Across all twelve seasons, the proxy reproduces the true FG%-by-range curve to within 2.3–3.5 percentage points, with a total variation distance of

0.050–0.113 and an agreement bound of 83.8–86.4%. Validation also revealed that we had been applying the 14-second reset in seasons preceding the 2018–19 rule change and correcting this halved the distributional error in those five seasons and restored the proxy–truth FG% curve correlation from approximately zero to 0.91–0.97, while leaving later seasons identical shot-for-shot. One bias remains, which is that the clock starts at the triggering event rather than the inbound touch, so possessions following a made basket are charged for inbound time that never elapsed, leaving the 24–22 second range under-populated (0.5–0.8% of attempts against a true 3.0–5.5%).

The estimated shot clock is divided into three buckets: early (>18 seconds), middle, and late (<4 seconds), since teams often take harder shots when rushed or at the end of the clock versus when they can patiently create an advantage. Shot distances are also grouped into predefined distance ranges: 0–3 ft, 3–10 ft, 10–16 ft, 16–23 ft, 23+ ft) to more clearly separate types of shots. An initial approach used defender distance; however, because Savant only tracked defender distance until January 24, 2016, the analysis instead relies on the proxy shot clock, contest score, and other features.

Figure 4 shows the distribution of estimated shot clock values for made and missed shots.

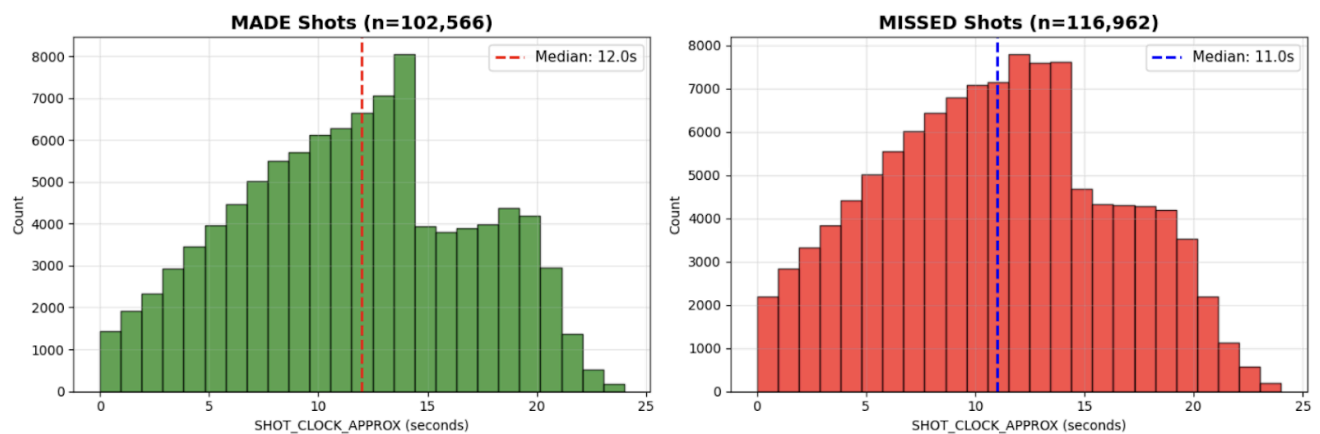


Figure 4. Distribution of shot clock values for made (left) and missed (right) shots in the 2025 season. The drop at 14 is because on some dead balls, the shot clock resets to 14 instead of 24.

Using the estimated shot clock and the raw data, features are constructed to capture four broad aspects of a shot: location, game context, shot characteristics, and historical data. Location features describe where the shot was taken, and contextual features represent the clock and possessions. Shot characteristics are action type, and historical data features are derived from prior-season statistics for the opposing defense. The complete list of these features and their explanations is provided in Appendix A. Two features in the data are deliberately excluded from the models: the shooter's prior-season shooting stats, and the team's identity (teamTricode). The point of shot quality is to predict the probability that a league-average shooter makes a shot, so a model that is aware of who is shooting through the player's history or a team indicator doesn't align with our goals. Excluding these features costs 0.003 AUC within a season. Note that prior-season opponent-defense features are retained since that only affects the difficulty of the shot, not who is shooting it.

The task is treated as a supervised learning problem: predict the binary outcome from all contextual features. The 2024–25 season data are split into training and test sets (80%, 20%). Several models are then trained: logistic regression, k-nearest neighbors, tree-based models such as random forests and gradient boosting, and more advanced libraries like XGBoost, CatBoost, and LightGBM. Nine baseline models are trained with fixed hyperparameters, and the other three (XGBoost, CatBoost, and LightGBM) are further tuned using Bayesian optimization via Optuna with a TPE sampler, 10 trials per model, optimizing for ROC AUC with 3-fold stratified cross-validation. The full set of hyperparameters for all models is provided in Appendix B. Model performance is evaluated using log loss, AUC, Brier score, and accuracy. I further examine calibration curves and confusion matrices to conduct a deeper error analysis. The

best-performing model is then used to produce probabilities for every shot. These probabilities are scaled by shot value to compute expected points, which are then aggregated to generate player- and team-level metrics.

Note that this will differ from real points scored since free throws are not included. Because the model doesn't count free throws, team win totals are computed similarly. A team is credited with a win in each game where it outscored the opponent on field goals, with the thirty games that ended in a tie split half and half between the two teams. The real win-loss record is also reported alongside it.

To further validate the approach, cross-season testing is performed. Instead of splitting 80%, 20% over a single season, I train on earlier seasons and test on the following. Two temporal validation strategies are used. First, a cumulative approach trains on all prior seasons and tests on the next (e.g. training on 2014–2018 and testing on 2019), repeated eight times ending with 2024–2025. Second, a year-to-year test trains on a single season X and tests on season $X+1$, repeated across five iterations. In both setups, prior-season statistics such as opponent defensive ratings are merged using only data from the season preceding the training period, preventing leakage. These tests mirror the real-world use case where a model is trained on the current season to predict future ones. If these tests give similar results to the single-season tests, it would show that the model works well across seasons. Three additional tests are performed using the model. First, feature importance scores are computed based on total gain across all LightGBM trees. Second, the model is retrained on reduced sets of features grouped into three tiers to assess the contribution of each group. Third, expected points for each shot are computed by multiplying the probability by the shot value (two or three points), and then these are aggregated to produce SPOE (shooting performance over expected), which is actual points minus expected points.

$$E[\text{points} \mid x] = v(x) \cdot P(\text{make} \mid x), \text{ where } v(x) \in \{2, 3\}.$$

$$\text{SPOE} = \sum_i v(x_i) (1\{\text{make}_i\} - P(\text{make} \mid x_i))$$

To assess whether SPOE reflects skill or variance, each player's per-shot SPOE was computed separately by season and correlated across consecutive seasons, and the sampling variance of SPOE was derived:

$$\text{Var}(\text{SPOE}) = \sum_i v(x_i)^2 \cdot p_i(1 - p_i).$$

The standard error is $\text{SE} = \sqrt{\text{Var}(\text{SPOE})}$, and each player-season is judged against a 95% interval ($\pm 1.96 \cdot \text{SE}$), which is the cutoff used to count the number of significantly above- and below-expectation players reported in Results. To give these intervals a scale: a two-point attempt at the league rate ($p \approx 0.55$) contributes 1.0 point² of sampling variance, while a three-point attempt ($p \approx 0.36$) contributes ≈ 2.1 . From the 2024–25 mix of roughly 58% twos and 42% threes, the average is ≈ 1.45 points² per shot, giving $\text{SE} \approx 1.2\sqrt{n}$: about 12 points at the 100-attempt threshold and about 45 for the highest-volume shooters (~ 1500 attempts), so the 95% luck bands span roughly ± 25 to ± 90 points.

The same variance also yields a second, independent estimate of the skill-versus-luck split: $\sigma^2_{\text{true}}/\sigma^2_{\text{observed}} = 0.575$, which agrees with the year-over-year correlation of 0.570, as mathematically expected.

Results

Model accuracy

Figure 5 summarizes the performance of each model, evaluated using AUC, accuracy, Brier score, and log loss.

model	cv_roc_auc	cv_log_loss	cv_accuracy	test_roc_auc	test_log_loss	test_brier	test_accuracy	best_threshold	best_params
voting_ensemble	N/A	N/A	N/A	0.665227	0.633971	0.222733	0.6339	0.47	Voting: ['lightgbm', 'hist_gb', 'gb', 'xgboost...]
xgboost	0.6631 ± 0.0020	0.6353 ± 0.0010	0.6310 ± 0.0004	0.664928	0.633898	0.222721	0.634742	0.49	{'n_estimators': 353, 'max_depth': 7, 'learnin...
lightgbm	0.6625 ± 0.0020	0.6355 ± 0.0009	0.6307 ± 0.0004	0.664629	0.63396	0.222772	0.635243	0.49	{'n_estimators': 563, 'max_depth': 8, 'learnin...
hist_gb	0.6620 ± 0.0018	0.6359 ± 0.0008	0.6313 ± 0.0001	0.664538	0.634413	0.222917	0.634947	0.5	{'max_iter': 339, 'max_depth': 12, 'learning_r...
gb	0.6621 ± 0.0020	0.6358 ± 0.0009	0.6317 ± 0.0005	0.664296	0.634615	0.22298	0.635061	0.48	{'n_estimators': 433, 'max_depth': 4, 'learnin...
catboost	0.6625 ± 0.0015	0.6357 ± 0.0007	0.6315 ± 0.0010	0.664169	0.634643	0.222974	0.633011	0.47	{'iterations': 850, 'depth': 5, 'learning_rate...
rf	0.6614 ± 0.0020	0.6360 ± 0.0009	0.6311 ± 0.0008	0.662954	0.634914	0.223132	0.633535	0.5	NaN
extra_trees	0.6597 ± 0.0023	0.6369 ± 0.0009	0.6312 ± 0.0007	0.661981	0.635687	0.223454	0.63374	0.52	NaN
log_reg	0.6583 ± 0.0022	0.6379 ± 0.0009	0.6302 ± 0.0009	0.660006	0.636618	0.223887	0.633148	0.51	NaN
mlp	0.6590 ± 0.0023	0.6372 ± 0.0012	0.6307 ± 0.0011	0.658161	0.636625	0.223856	0.63374	0.5	NaN
adaboost	0.6414 ± 0.0015	0.6485 ± 0.0007	0.6262 ± 0.0015	0.645118	0.646683	0.228116	0.631736	0.53	NaN
naive_bayes	0.6399 ± 0.0013	7.8493 ± 0.0943	0.6187 ± 0.0016	0.643305	7.722906	0.373952	0.621897	0.9	NaN
knn	0.6263 ± 0.0009	0.7330 ± 0.0008	0.5988 ± 0.0006	0.627158	0.730909	0.240448	0.60773	0.62	NaN

Figure 5. Model performance comparisons across algorithms. Metrics are shown for both cross-validation (cv_) and testing (test_)

The test scores are slightly higher than the cross-validation scores, indicating limited overfitting. The top three models are effectively tied: the voting ensemble, XGBoost, and LightGBM finish within 0.0006 AUC of one another (0.6652, 0.6649, and 0.6646), well inside the cross-validation standard deviation of ± 0.002 . Each leads on a different metric: the ensemble on AUC, XGBoost

on log loss and Brier score, LightGBM on accuracy. Because the differences are within noise, LightGBM is used as the scoring model for all downstream analyses.

To further solidify and validate these results, Figures 6 and 7 show the year-to-year temporal validation results. All performance indicators remain largely consistent across the seasons.

Fold	Train N	Test N	Features	AUC	Log Loss	Brier	Accuracy
2020 → 2021	188,116	190,983	55	0.655975	0.639131	0.225169	0.628904
2021 → 2022	190,983	216,722	55	0.663149	0.634117	0.222757	0.636437
2022 → 2023	216,722	217,219	55	0.667191	0.634281	0.223011	0.632597
2023 → 2024	217,219	218,702	55	0.663853	0.635592	0.223613	0.630954
2024 → 2025	218,702	219,528	55	0.661397	0.635873	0.223640	0.632115

Figure 6. Year-to-year results for LightGBM, showing performance trained on season X and testing on X+1 for the past 5 years.

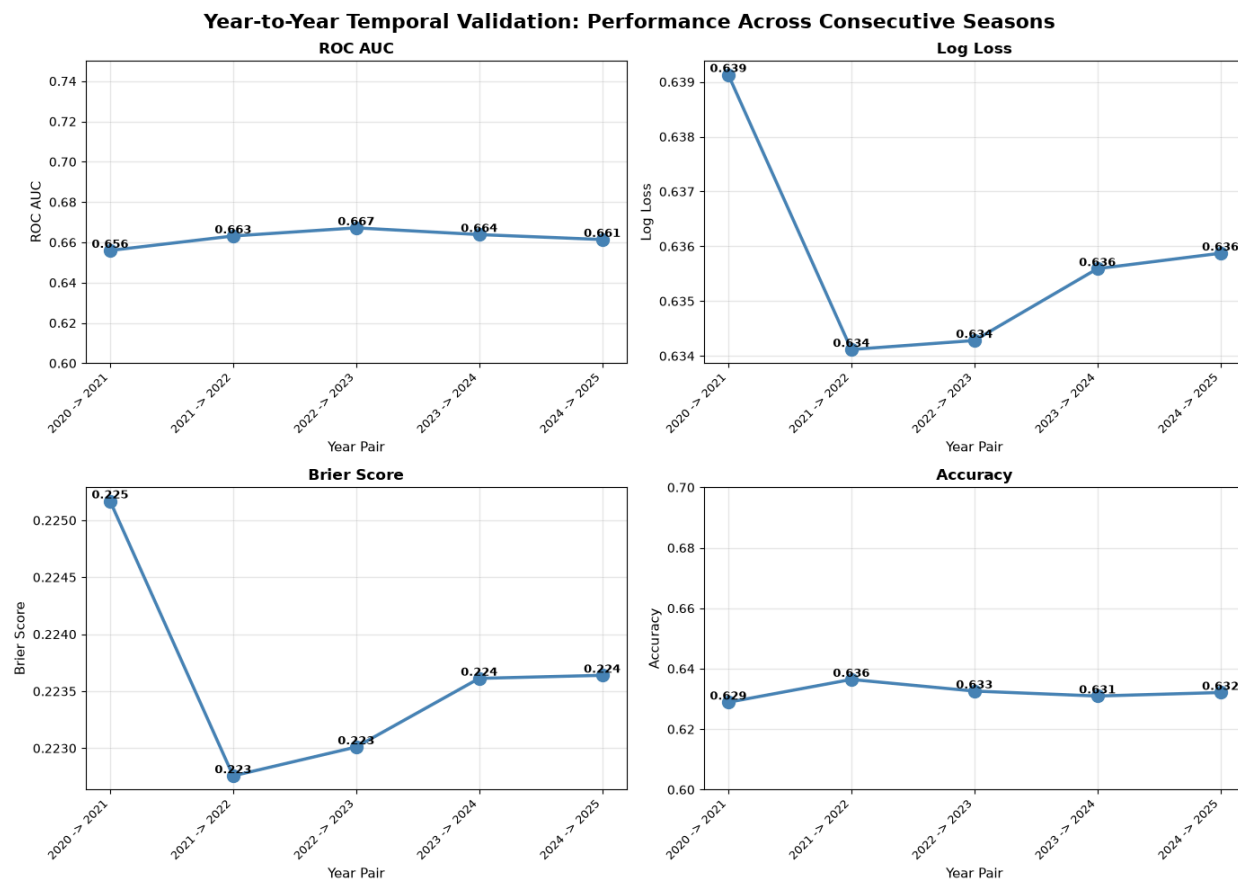


Figure 7. Year-to-year performance indicators plotted on graphs.

Analysis of important features

Figure 8 shows the top 30 features by importance.

Feature	Importance
shot_angle_rad	3380
xLegacy	3340
yLegacy	3146

period_sec_remaining	2916
ABS_TIME	2710
period_sec_elapsed	2654
opp_def_3pt_pct	2243
clock_x_distance	2146
game_sec_remaining_reg	2031
opp_def_mid_pct	1969
opp_def_rim_pct	1870
SHOT_CLOCK_APPROX	1652
opp_def_fg_pct	1627
opp_def_shot_type_pct	1535
contest_x_shot_clock	1478
contest_x_distance	1294
shotDistance	1201
rhythm_x_distance	620
clock_x_paint	600
shot_distance_log	541
contest_score	346
is_layup	329
is_dunk	326
is_finger_roll	322
is_driving	312
shot_transition	300
subType_Driving Layup Shot	294
location_h	277
is_tip	269
is_bank	255

Figure 8. Top 30 features ranked by LightGBM importance. Higher results indicate features that were reduced more frequently during tree construction.

To assess the relative contribution of feature groups, the top 30 features were grouped into three tiers based on their role (Figures 9, 10, 11). Figure 12 shows the results.

Feature	Importance	Description
shot_angle_rad	3,380	Angle to basket from coordinates
xLegacy	3,340	Court x-coordinate (left/right)
yLegacy	3,146	Court y-coordinate (baseline to half-court)
period_sec_remaining	2,916	Seconds remaining in the current period
ABS_TIME	2,710	Absolute game time in seconds
period_sec_elapsed	2,654	Seconds elapsed in the current period

Figure 9. Top features grouped as shot location and game time

Feature	Importance	Description
opp_def_3pt_pct	2,243	Opponent's 3PT defense rating
clock_x_distance	2,146	Shot clock × distance interaction
game_sec_remaining_reg	2,031	Game seconds remaining (regulation)
opp_def_mid_pct	1,969	Opponent's mid-range defense rating
opp_def_rim_pct	1,870	Opponent's rim defense rating
SHOT_CLOCK_APPROX	1,652	Approximate shot clock
opp_def_fg_pct	1,627	Opponent's overall FG% allowed
opp_def_shot_type_pct	1,535	Opponent defense matching shot type
contest_x_shot_clock	1,478	Contest × shot clock interaction
contest_x_distance	1,294	Contest × distance interaction

shotDistance	1,201	Distance from basket in feet
--------------	-------	------------------------------

Figure 10. Top features related to game context and opponent defense.

Feature	Importance	Description
rhythm_x_distance	620	Rhythm shot × distance interaction
clock_x_paint	600	Shot clock × paint shot interaction
shot_distance_log	541	Log-transformed distance
contest_score	346	Heuristic contest difficulty
is_layup	329	Shot is a layup
is_dunk	326	Shot is a dunk
is_finger_roll	322	Shot is a finger roll
is_driving	312	Shot is a driving action
shot_transition	300	Transition-flavored shot
subType_Driving Layup Shot	294	Categorical: driving layup
location_h	277	Home game (categorical)
is_tip	269	Tip-in
is_bank	255	Bank shot

Figure 11. Features associated with shot characteristics.

Figure 12 shows the results from running the model on these three tiers. I observe that AUC rises from 0.646 to 0.649, stays the same at 0.648 (since it's within the error bound) across the three configurations. Also keep in mind that the full model uses all 55, not just these 22 top features. So most of the predictive value comes from a small set of defensive- and clock-context features, while the others contribute little.

Configuration	Features	AUC	Log Loss	Brier	Accuracy
Top-Tier Only	6	0.645816	0.649098	0.228936	0.625860
Top + Second Tier	17	0.648806	0.646232	0.227718	0.629140
All Tiers	22	0.647799	0.646266	0.227767	0.629026

Figure 12. Performance of the LightGBM model trained using progressively bigger subsets of features.

The first configuration doubles as the naive baseline a skeptical reader might propose since it only knows where on the floor the shot was taken and when in the game. It already reaches an AUC of 0.646, which is most of the distance from chance (0.500) to the full model (0.665). The engineered context, like shot clock, contest score, action type, and opponent defense, adds the remaining 0.02. What would be different if the baseline performed nearly as well? At the player level, little, since SPOE's skill/variance decomposition depends on sample sizes rather than model sophistication, and the ~75% of player-seasons indistinguishable from zero is stable across specifications. At the team level, more since the added context sharpens expected points enough to change several teams' expected-win totals by multiple wins. The contribution of the engineered features is therefore not the headline AUC, which location and time largely determine, but the fidelity of the aggregated expected-points estimates.

Expected points analysis

Aggregating expected points to the player level, SPOE distributions (Figures 13 and 14) were approximately centered on zero (median percent difference -1.2%), with a median absolute

deviation of about 23 points. The largest positive deviations belong to established elite tough shot-makers: Nikola Jokić (+294.2 over 1,364 shots), Kevin Durant (+211.6, 1,124 shots), and Shai Gilgeous-Alexander (+181.5, 1,656 shots). Jokić's standard error is about 44 points, so luck explains at most ± 87 . So his +294 is roughly seven standard errors above expectation. A league-average shooter has about a 1 in 40 billion chance of achieving these results.

The largest negative deviations belong almost entirely to first- and second-year players: Alexandre Sarr (−149.9, 828 shots), Stephon Castle (−142.5, 988 shots), and Bilal Coulibaly (−113.1, 624 shots), which is consistent with young players taking reasonable shots but converting them below the league-average rate.

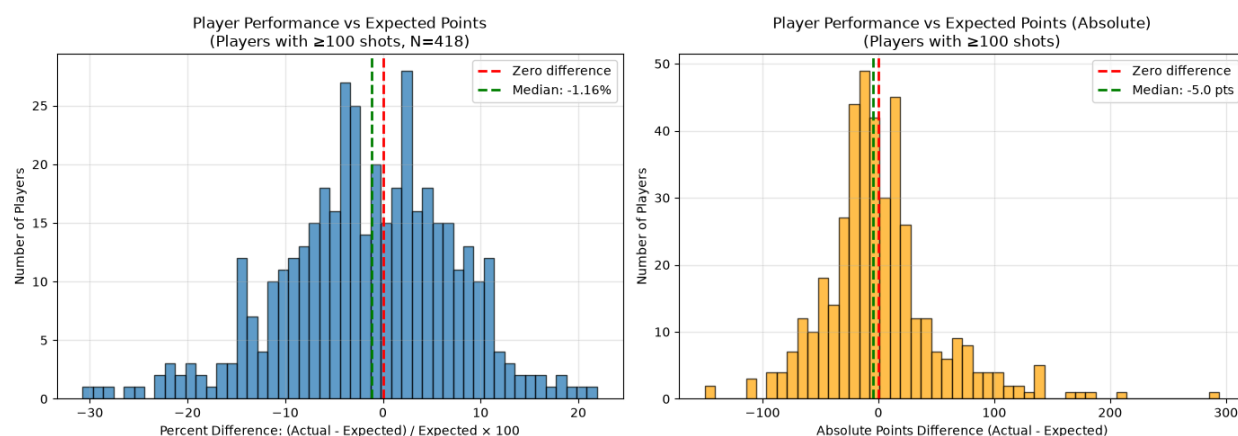


Figure 13. Players' results vs expected points measured by absolute and percent difference.

Figure 14 restricts the analysis to high-volume shooters, reducing the influence of small-sample variance.

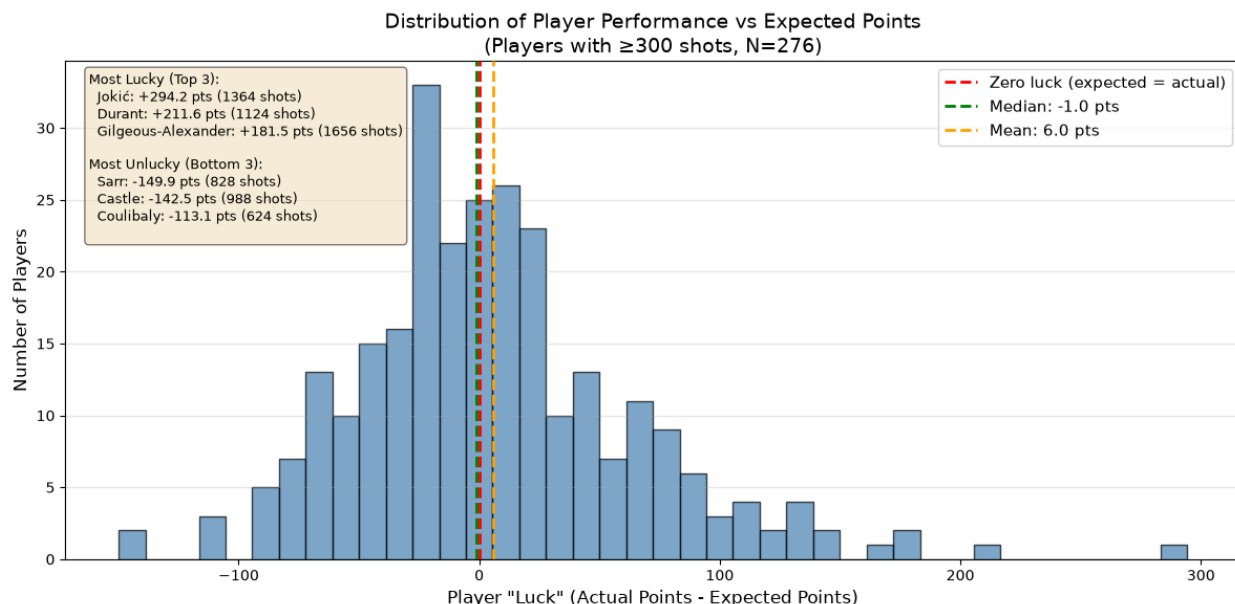


Figure 14. Player's results vs expected points measured by absolute difference for high-volume shooters (≥ 300 shots).

Per-shot SPOE was compared across consecutive seasons for every player. Among those with at least 200 attempts in both seasons, the year-over-year correlation was 0.570, rising monotonically to 0.707 for players with at least 800 attempts. Partitioning the observed variance between players into true between-player variance and sampling variance gives a reliability of 0.575. Applying 95% intervals to individual players across 4,691 player-seasons with at least 100 attempts, 552 were significantly above expectation and 629 significantly below, while the remaining 3,510 (approximately 75%) could not be distinguished from zero.

At the team level, summing expected points for and against each team and plugging those into a Pythagorean formula (Figure 15) produced expected win totals.

A scatterplot of expected wins against both the real win-loss record and field-goal-only wins (Figure 16) shows a clear positive relationship: $r = 0.65$ against the real record, 0.79 against field-goal-only wins, with mean absolute errors of 8.8 and 6.3 , respectively. Several teams are well off the line, and for four of them the two measures disagree in sign.

$$WinPct = \frac{PointsScored^{14}}{PointsScored^{14} + PointsAllowed^{14}}$$

Figure 15. Pythagorean win-loss formula

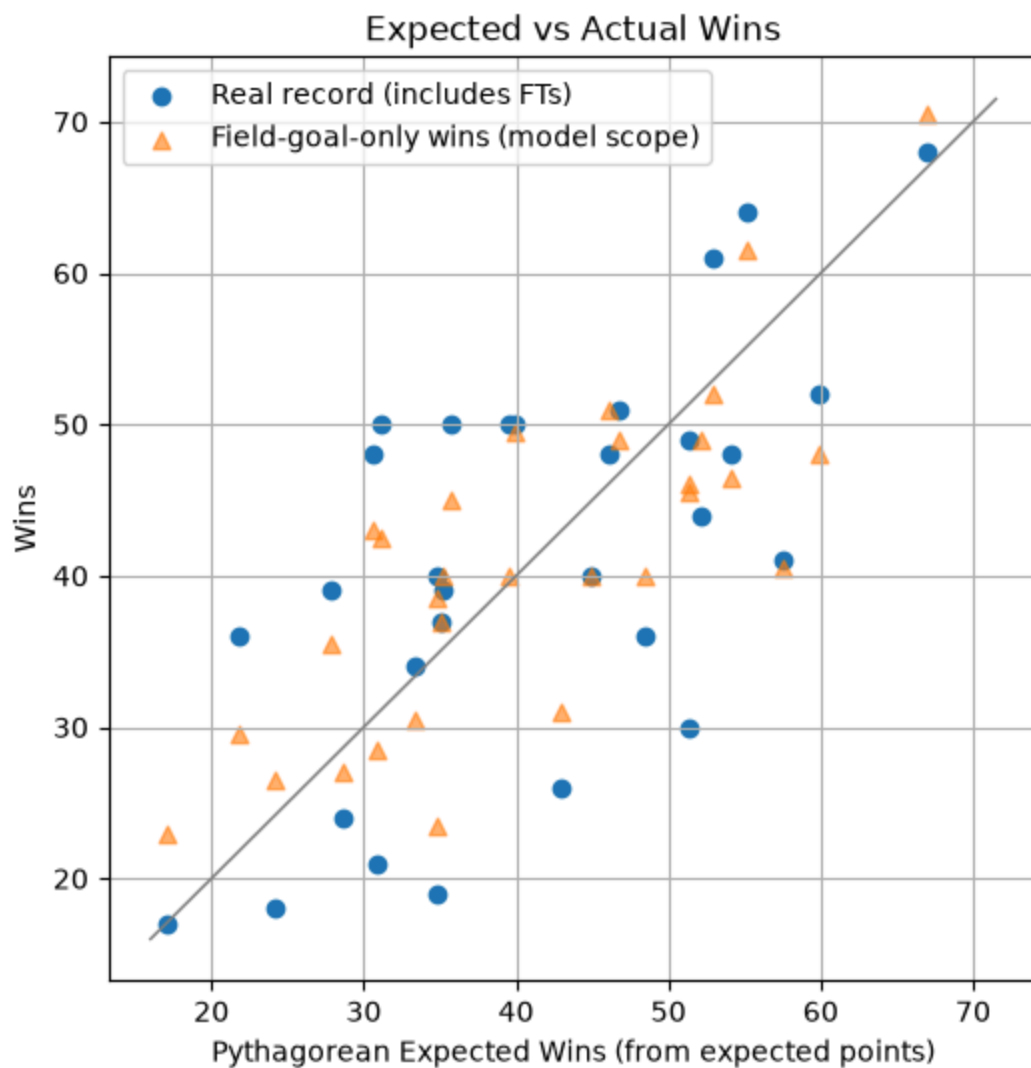


Figure 16. Scatterplot of expected vs actual and field-goal-only wins of NBA teams from the 2024-2025 season

The table (Figure 17) reports expected points, expected wins, and the difference against both win measures. Across the thirty teams, the gap against the real record correlates -0.88 with team-level SPOE. The Raptors (+21.4), Nets (+17.0), Magic (+16.5) and Hornets (+15.8) generated shot profiles worth far more wins than they achieved, meaning that their rosters converted well below league average based on their shot quality, while the Lakers (-18.9),

Bucks (-17.4), Pacers (-14.2) and Suns (-14.2) won far more than their shot profiles alone would deliver, showing above-average shot-making.

teamTricode	pts_scored_exp	pts_allowed_exp	pyth_wins_exp	fg_only_wins	diff_vs_fg_only	wins_actual	diff_vs_actual
OKC	8309.65	7465.61	67.04	70.5	-3.46	68	-0.96
HOU	8217.68	7654.76	59.84	48	11.84	52	7.84
ORL	7557.34	7110.58	57.5	40.5	17	41	16.5
CLE	8337.74	7919.03	55.18	61.5	-6.32	64	-8.82
GSW	7963.16	7594.02	54.14	46.5	7.64	48	6.14
BOS	7927.93	7596.37	52.91	52	0.91	61	-8.09
DET	7995.62	7683.33	52.15	49	3.15	44	8.15
TOR	8314.9	8013.01	51.38	45.5	5.88	30	21.38
MIN	7924.92	7638.82	51.33	46	5.33	49	2.33
POR	8118.68	7908.16	48.46	40	8.46	36	12.46
NYK	8016.41	7855.16	46.79	49	-2.21	51	-4.21
MEM	8302.67	8155.46	46.11	51	-4.89	48	-1.89
ATL	8387.23	8271.83	44.96	40	4.96	40	4.96
BKN	7636.69	7584.72	42.96	31	11.96	26	16.96
LAC	7574.19	7602.66	39.92	49.5	-9.58	50	-10.08
DEN	8096.83	8139.66	39.49	40	-0.51	50	-10.51
IND	7907.61	8053.57	35.78	45	-9.22	50	-14.22
CHI	8396.62	8568.44	35.23	40	-4.77	39	-3.77
MIA	7742.38	7905.35	35.06	37	-1.94	37	-1.94
SAC	7710.77	7878.96	34.85	38.5	-3.65	40	-5.15
CHA	7840.63	8014.09	34.77	23.5	11.27	19	15.77
SAS	8175.11	8399.13	33.33	30.5	2.83	34	-0.67
LAL	7627.55	7900.86	31.1	42.5	-11.4	50	-18.9
NOP	7983.49	8278.01	30.82	28.5	2.32	21	9.82
MIL	7800.09	8094.33	30.6	43	-12.4	48	-17.4
PHI	7803.92	8157.57	28.67	27	1.67	24	4.67
DAL	7807.25	8187.84	27.82	35.5	-7.68	39	-11.18
WAS	7873.15	8378.25	24.2	26.5	-2.3	18	6.2
PHX	7481.9	8043.79	21.83	29.5	-7.67	36	-14.17
UTA	7769.67	8548.38	17.05	23	-5.95	17	0.05

Figure 17. Team expected points and expected wins, compared with field-goal-only wins and actual wins.

Figure 18 shows that teams getting more open looks tend to win more games, though the only moderate correlation indicates that other factors also contribute.

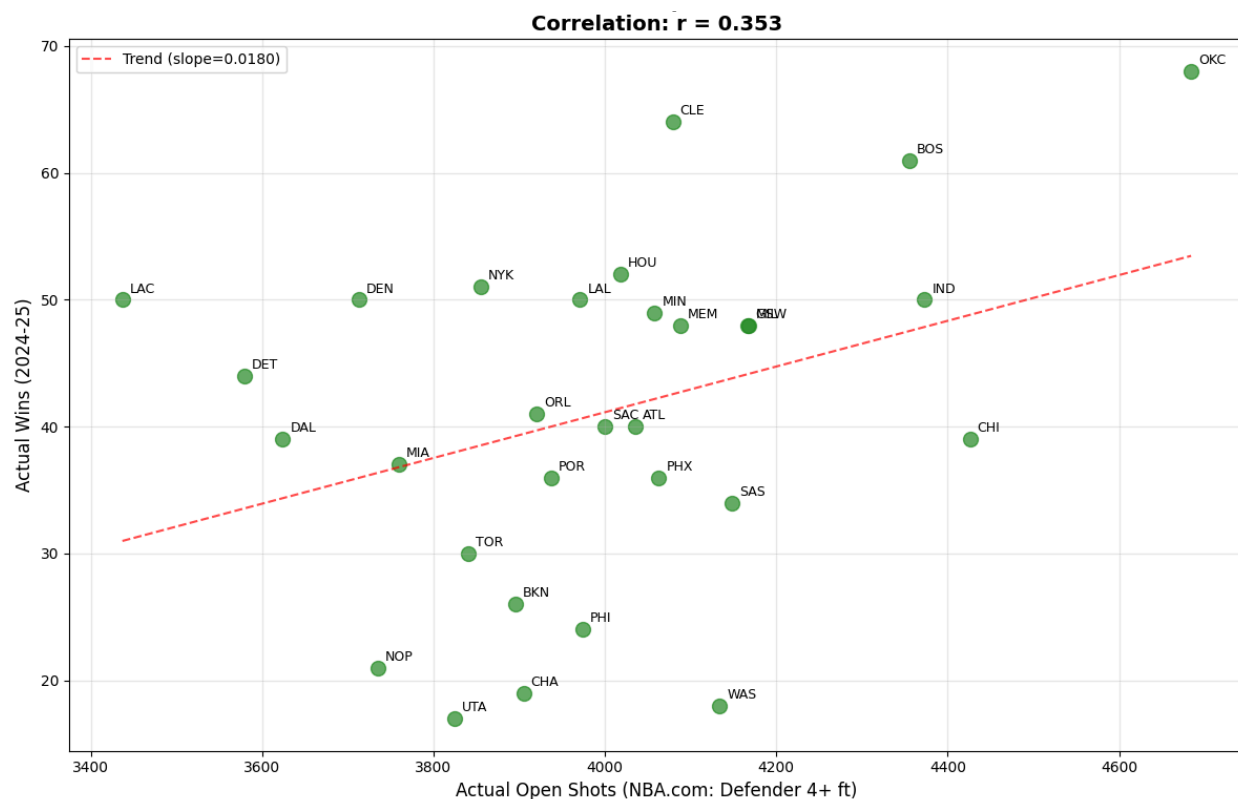


Figure 18. Open shots vs wins

Figure 19 presents the confusion matrix for the LightGBM model on the test set at the 0.49 decision threshold. The model correctly classifies 83.2% of missed shots (19,466 of 23,393) but only 41.1% of made shots (8,425 of 20,513), for an overall accuracy of 63.5% against a majority-class baseline of 53.3%. Because misses outnumber makes, the model is biased toward predicting a miss, producing far more false negatives (12,088) than false positives (3,927). This asymmetry follows from thresholding a probability at a fixed cutoff and is not a defect in the probabilities themselves; as Figure 20 shows, the underlying predictions remain well calibrated.

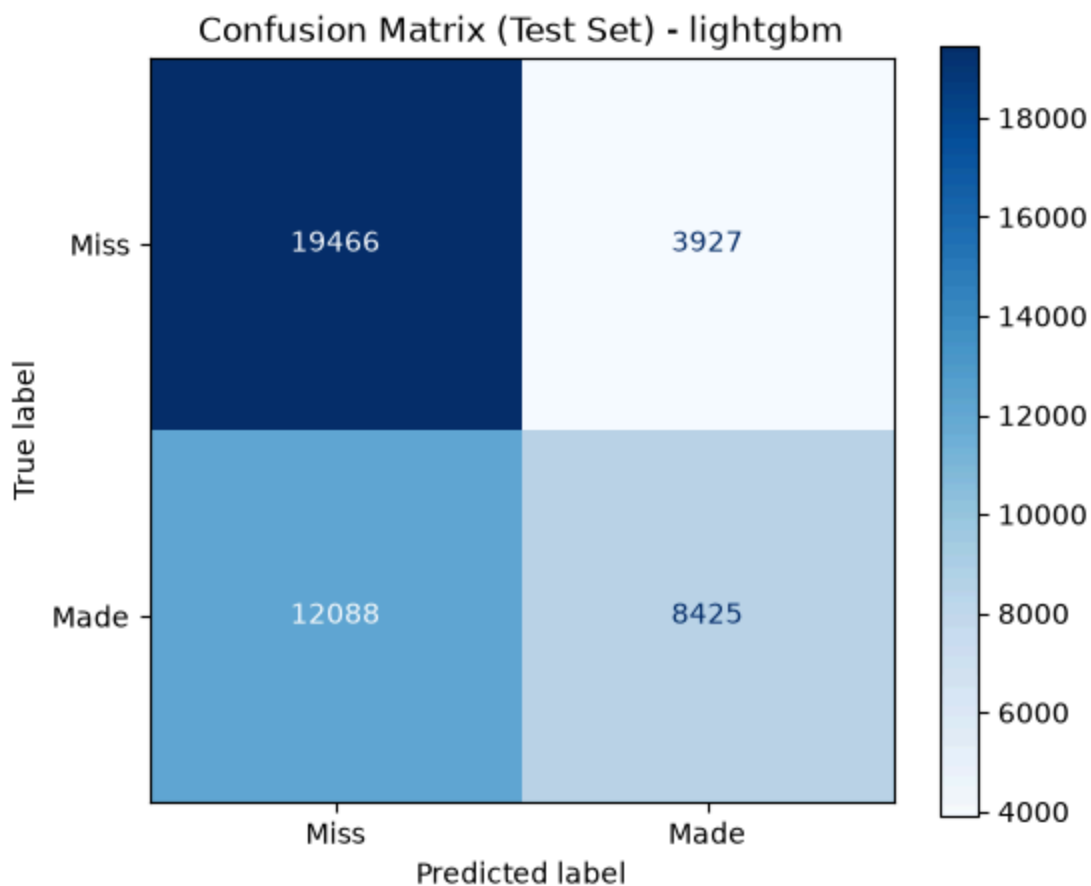


Figure 19. Confusion matrix for the LightGBM model.

Figure 20 displays the calibration curve for the LightGBM model, comparing predicted probability to observed frequencies across probability bins. The curve lies close to the $y=x$ reference line, indicating that when the model predicts, for example, a 65% chance of a make, shots actually go in about 65% of the time. This means that in addition to being accurate overall, it works well for all types of shots.

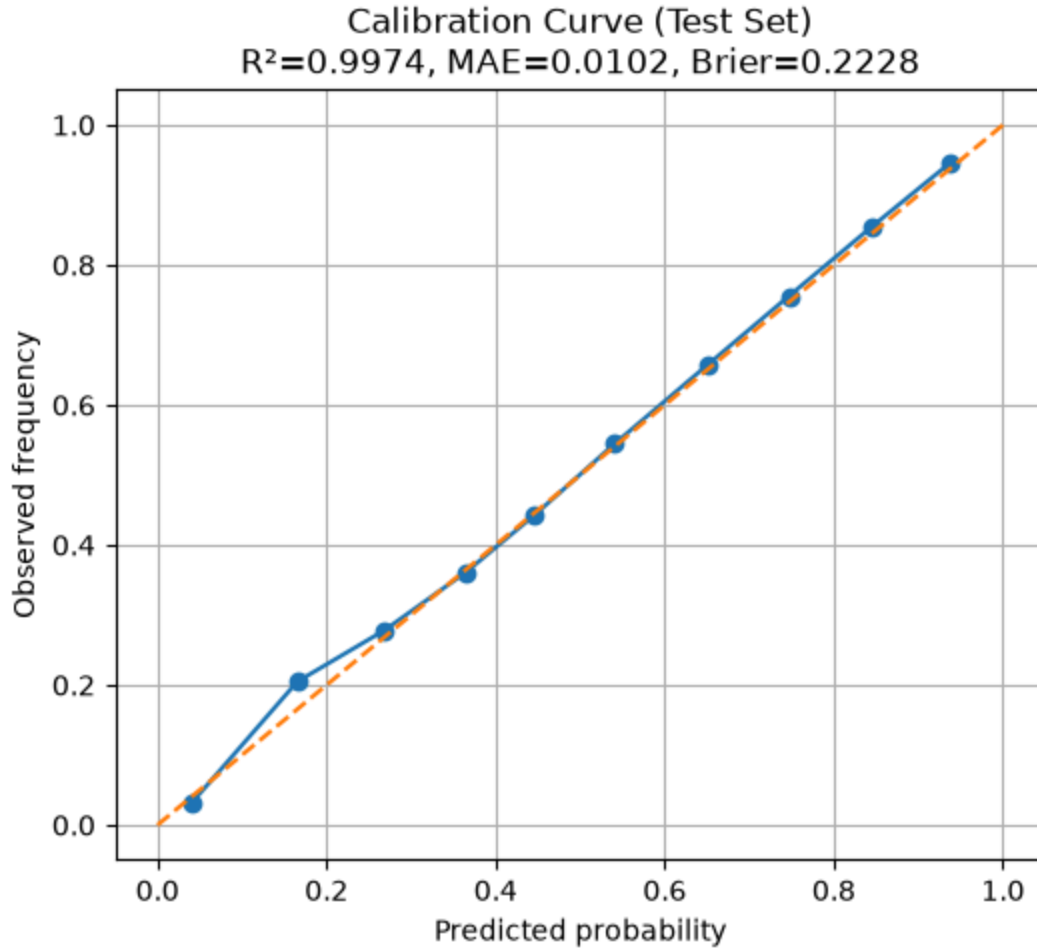


Figure 20. Calibration curve for the LightGBM model.

Discussion

These results demonstrate that shot quality, measured using only public data, is sufficient to separate a team's offensive strategy from individual player skill, and to link both to winning. But the results also indicate that shot outcome prediction using public data has an inherent performance ceiling. All of the advanced models achieved nearly identical AUC scores (LightGBM: 0.6625 ± 0.0020 , XGBoost: 0.6631 ± 0.0020 , CatBoost: 0.6625 ± 0.0015), suggesting that they have reached a shared ceiling. This convergence suggests that the

remaining unexplained variance in shot outcomes is largely attributable to factors not observable in public data. Achieving higher performance would require access to player tracking or ball trajectory data. This AUC of 0.665 is below the 0.843 reported by Shah and Romijnders (2016), but their model relies on proprietary ball-tracking data unavailable for public and general use. The contribution of this paper is to demonstrate that publicly available features can produce an adequate shot quality model without proprietary infrastructure.

At the player level, deviations between actual and expected scoring reflect a combination of variance and persistent performance differences. For high-volume shooters, persistent deviations from expectation likely reflect underlying differences in shot-making ability that are not fully captured by a model that treats shooters as league-average from a given location. In contrast, for players with small sample sizes, measured SPOE may be caused by short-term variance. As a result, the model may underestimate true outliers and attribute their persistent overperformance to randomness rather than skill.

These effects can be separated using the twelve-season analysis. Shooting over expectation does persist across seasons, and it persists more clearly the more shots a player takes. Roughly three-fifths of the observed spread between shooters reflects genuine differences in shot-making and two-fifths reflects chance. The practical implication is that SPOE is real but slow to measure since at typical shot volumes most players cannot be reliably distinguished from league average, and only sustained deviation across multiple seasons, or very high shot counts within one, constitutes evidence of shot-making ability rather than a hot streak.

At the team level, expected wins from shot quality alone correlate 0.65 with the real record (Figure 16), explaining roughly 42% of the variance in win totals. The deviations from the identity line are not noise because the model assumes league-average shooting; a team's

distance from the line measures how far its shooters converted above or below that expectation, so the two components together decompose team success into shot generation and shot-making. Several teams illustrate this separation clearly.

The two win columns in Figure 17 separate two different causes. Free-throws, which the model does not use, account for much of the gap against the real win-loss record. A team's net free-throw differential correlates -0.65 with its gap against the real record, and its shooter execution (team SPOE) correlates -0.88. Toronto is the clearest case: its shot profile was worth roughly 51 wins, but it won 30, and its shooters converted 518 points below expectation which was the largest execution deficit in the league. Phoenix is the opposite: a shot profile worth roughly 22 wins carried to 36 by shooters who converted 456 points above expectation (notably Kevin Durant and Devin Booker are prolific tough shot makers). Boston combines good shooting (+355) with a positive free-throw differential (+123), so a model blind to both underestimates it by about eight wins.

Analysis of specific classification errors further clarifies the behavior of the model. The confusion matrix (Figure 19) shows a greater number of false negatives than false positives, reflecting the imbalance in shot outcomes, where there are more misses than made shots. As a result, the model is more inclined to predict misses, even for shots with relatively high make probabilities. Since this asymmetry is consistent with the distribution of makes and misses, it doesn't necessarily indicate a failure of calibration, as confirmed by the near-linear calibration curve (Figure 20).

The following discussion considers potential applications of these findings. The results suggest that shot quality is a large part of true team strength, but not all of it. Teams whose expected wins are close to their actual records appear to have played roughly to the quality of their shots, meaning they generated the kinds of looks the model rates as efficient and then made them at a league-average rate.

A team far below its expected wins (Toronto, Charlotte, Orlando) has a good offensive process and a roster that cannot convert it: the indicated remedy is shooting talent, not a new system. A team far above (Phoenix, the Lakers) is being carried by shot-making over a weak process, which is the profile most exposed to decline and injury. A front office might hire a new coach or change their scheme in this situation. At the player level, a negative SPOE identifies conversion below league average from that player's shot locations; for young players this is a development baseline rather than a verdict, and the year-over-year analysis shows roughly forty percent of a single season's deviation is noise in any case.

Conclusion

This paper approaches shot quality as a conditional-probability problem: estimating $P(\text{make} \mid \text{shot features})$ from only public data. This paper uses models, including LightGBM, to generate shot-making probabilities that are accurate and consistent over multiple seasons. These probabilities are then used to identify players who under- or over-perform relative to expectation. This means that it is a useful tool for separating game plan from shooting performance. Finally, we used team-wide shot quality to analyze winning over a full season: the shot profiles teams generate explain roughly 42% of the variance in their win totals, and the deviations measure how far their shooters converted above or below league average.

There are also some areas for improvement. I am not including free throws, which may hurt teams that rely heavily on drawing fouls. Also, the lack of full player-tracking data requires defender positioning and other features to be approximated only via rough proxies. The features are also calibrated to the modern NBA and therefore may not generalize well to earlier eras or other levels of play. Future work could address these issues by building multi-season models that observe players and teams evolving or extending from basic expected points to a continuous possession value. As player-tracking becomes more accessible, those features could be incorporated to further this analysis.

Acknowledgements

Thanks to Michael Yetman for his help in drafting and editing this paper.

References

- Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2014). Predicting points and valuing decisions in real time with NBA optical tracking data. *MIT Sloan Sports Analytics Conference*. https://www.lukebornn.com/papers/cervone_ssac_2014.pdf
- Cervone, D., D'Amour, A., Bornn, L., & Goldsberry, K. (2016). A multiresolution stochastic process model for predicting basketball possession outcomes. *Journal of the American Statistical Association*, 111(514), 585–599.
<https://doi.org/10.1080/01621459.2016.1141685>
- Chang, Y.-H., Maheswaran, R., Su, J., Kwok, S., Levy, T., Wexler, A., & Squire, K. (2014). Quantifying shot quality in the NBA. *MIT Sloan Sports Analytics Conference*.

<https://www.semanticscholar.org/paper/Quantifying-Shot-Quality-in-the-NBA-Chang-Maheswaran/2ef041e6d5b83979d37cb0526b7bcc9f63c5b59e>

Madhavan, S., Van Negri, E., & Yu, L. A. (2017). *Nothing but neural nets: Predicting NBA basketball shot success* (Final project report, CS229). Stanford University.

<https://cs229.stanford.edu/proj2017/final-reports/5218669.pdf>

Oughali, M. S., Bahloul, M., & El Rahman, S. A. (2019). Analysis of NBA players and shot prediction using random forest and XGBoost models. *2019 International Conference on Computer and Information Sciences (ICCIS)*.

<https://doi.org/10.1109/ICCISci.2019.8716412>

Shah, R. C., & Romijnders, R. (2016). *Applying deep learning to basketball trajectories*. arXiv.

<https://arxiv.org/abs/1608.03793>

ShotQuality. (n.d.). *Shot Probability Model Whitepaper*. Retrieved March 26, 2026, from

<https://shotquality.com/>

Appendix A: Feature Definitions

This appendix is a list of all additional features and their purposes.

A1: Core shot and game context features

Feature	Description	Range
shotDistance	Distance from basket in feet	0–47 ft
SHOT_CLOCK_APPROX	Approximate shot clock when shot	0–24 sec

	taken	
xLegacy, yLegacy	Court coordinates (tenths of feet)	x: -250 to 250, y: 0 to 470
period	Game period (1–4, 5+ for OT)	1–7
shotValue	Point value of shot	2 or 3
contest_score	Heuristic difficulty score	-2 to +4
ABS_TIME	Absolute game time in seconds	0–2880+
actionType	Shot category (e.g., "2pt", "3pt")	Categorical
subType	Shot subtype (e.g., "pullup", "catch_and_shoot")	Categorical
contest_label	Shot openness classification	"likely_open", "borderline", "likely_contested"
teamTricode	Team abbreviation	Categorical (30 teams)
location	Home or away	"home", "visitor"

Figure A1. Engineered features all derived directly from raw play-by-play and shot location data.

Note that teamTricode was not used in the final model (see methods).

These features focus on basic improvements from raw data and are self-explanatory. One feature of particular interest is the contest_label variable. Among these, the contest_label acts as a heuristic for defensive pressure. It is constructed by combining the shot clock, shot type, and distance into a single value from -2 to 4, which is then converted into the categories: “open”, “borderline”, and “contested”.

A2: Court Location Indicator Features

Feature	Definition	Basketball Meaning
is_paint	shotDistance <= 8	Close-range shots near the basket
is_midrange	8 < shotDistance <= 22	Mid-range jumpers (dying art)

		in modern NBA)
is_corner3	$\text{abs}(x_{\text{Legacy}}) \geq 220 \ \& \ y_{\text{Legacy}} \leq 50 \ \& \ \text{shotValue} == 3$	Corner three-pointers (shortest 3PT shot)
is_above_break3	$\text{shotValue} == 3 \ \& \ \text{not is_corner3}$	Wing and top-of-key three-pointers

Figure A2. Binary indicators encoding shot location zones based on distance and Cartesian position.

A3: Shot Action Indicator Features

Feature	Pattern Matched	Basketball Meaning
is_pullup	"pullup", "pull-up", "pull up"	Shot off the dribble while moving
is_stepback	"step back", "stepback"	Creates separation by stepping away from defender
is_fadeaway	"fadeaway", "fade away", "turnaround fadeaway"	Jumping backward while shooting
is_turnaround	"turnaround" (not fadeaway)	Spin move into shot
is_driving	"driving"	Shot while attacking basket
is_floating	"floating", "floater", "runner"	High-arc shot over defender
is_cutting	"cutting", "cut"	Shot after moving without ball
is_tip	"tip"	Tipping ball near rim
is_putback	"putback", "put-back", "put back"	Shot immediately after offensive rebound
is_second_chance	is_tip OR is_putback	Any second-chance opportunity

Figure A3. Binary indicators encoding shot types derived from the actionType variable.

A4: Clock and Location Interaction Features

Feature	Definition	Basketball Insight
---------	------------	--------------------

clock_x_distance	shot_clock × distance	Continuous interaction
Transition (clock ≥ 18s)		Early offense, defense not set
is_transition_paint	clock ≥ 18 & distance ≤ 8	Fast break layup/dunk
is_transition_midrange	clock ≥ 18 & 8 < distance ≤ 22	Early midrange (rare)
is_transition_3pt	clock ≥ 18 & shotValue == 3	Transition three
Late Clock (clock ≤ 5s)		Shot clock pressure
is_late_clock_paint	clock ≤ 5 & distance ≤ 8	Desperation drive
is_late_clock_midrange	clock ≤ 5 & 8 < distance ≤ 22	Bailout midrange
is_desperation_3pt	clock ≤ 5 & shotValue == 3	Contested three to beat clock
is_heave_shot	clock ≤ 2 & distance ≥ 28	Half-court heave
Rhythm (8s ≤ clock ≤ 16s)		Set offense, good looks
is_rhythm_shot	8 ≤ clock ≤ 16	Ideal shot clock range
rhythm_x_distance	is_rhythm × distance	Rhythm shot by distance
Zone Interactions		
clock_x_paint	shot_clock × is_paint	Time pressure for paint shots
clock_x_corner3	shot_clock × is_corner3	Time for corner rotation
clock_x_above_break3	shot_clock × is_above_break3	Time for wing/top 3s

Figure A4. Interaction features attempt to represent time and distance in one variable by multiplying them.

A5: Prior-Season Shooter Features

Feature	Description	Source
prior_fg_pct	Player's FG% from prior season	Prior season totals
prior_fg3_pct	Player's 3PT% from prior season	Prior season totals
prior_fg2_pct	Player's 2PT% from prior season	Prior season totals
prior_shot_type_pct	Prior FG% matching shot type (3PT or	Conditional on shotValue

	2PT)	
prior_attempts	Total shot attempts in prior season	Prior season totals
prior_high_volume	Binary: prior_attempts >= 300	High-volume shooter flag

Figure A5. Prior season shooting percentages and totals for the shot taker. Note that these shooter prior-season features were not included in the final model (see methods).

A6: Prior-Season Defender Features

Feature	Description	Source
opp_def_fg_pct	Opponent's overall FG% allowed	Prior season defense
opp_def_rim_pct	Opponent's rim FG% allowed	Prior season defense
opp_def_3pt_pct	Opponent's 3PT% allowed	Prior season defense
opp_def_mid_pct	Opponent's mid-range FG% allowed	Prior season defense
opp_def_shot_type_pct	Opponent defense matching shot zone	Conditional on shot location

Figure A6. Previous season defensive performance for the shot defender.

A7: Shot Clock Bucket Features

Bucket	Range	Typical Situation
clock_0_4	0–4 sec	Shot clock desperation
clock_4_8	4–8 sec	Late clock, forced shots
clock_8_12	8–12 sec	Mid-possession
clock_12_16	12–16 sec	Set offense developing
clock_16_20	16–20 sec	Early offense
clock_20_24	20–24 sec	Transition/fast break

Figure A7. Discretized shot clock buckets.

Appendix B: Model Hyperparameters and Tuning

This appendix contains the hyperparameters for all 12 models used in this paper.

B1: Baseline Models

Nine baseline models are trained with fixed hyperparameters.

Model	Hyperparameters
Logistic Regression	solver="saga", max_iter=2000, C=0.5
k-Nearest Neighbors	n_neighbors=15, weights="distance"
Random Forest	n_estimators=500, max_depth=12, min_samples_leaf=5
Gradient Boosting	n_estimators=300, max_depth=5, learning_rate=0.1
MLP	hidden_layer_sizes=(128, 64), max_iter=500, early_stopping=True
Extra Trees	n_estimators=500, max_depth=12, min_samples_leaf=5
AdaBoost	n_estimators=200, learning_rate=0.1
Naive Bayes	GaussianNB (default, no tuning)
HistGradientBoosting	max_iter=500, max_depth=8, learning_rate=0.05

B2: Optuna-Tuned Models

Three models are tuned using Bayesian optimization via Optuna with a TPE (Tree-structured Parzen Estimator) sampler. Each model is tuned over 10 trials, optimizing for ROC AUC using 3-fold stratified cross-validation.

Model	Search Range
XGBoost	n_estimators [100–1000], max_depth [3–10], learning_rate [0.01–0.3], subsample [0.6–1.0], reg_alpha [1e-8–10], reg_lambda [1e-8–10]
CatBoost	iterations [100–1000], depth [4–10], learning_rate [0.01–0.3], l2_leaf_reg [1e-8–10], bagging_temperature [0–1]
LightGBM	n_estimators [100–1000], max_depth [3–12], learning_rate [0.01–0.3], num_leaves [16–128], min_child_samples [5–50]

B3: Final Model (LightGBM)

The best-performing model, LightGBM, has the following tuned hyperparameters:

Parameter	Value
n_estimators	563
max_depth	8
learning_rate	0.0117
num_leaves	84
subsample	0.668
colsample_bytree	0.626
reg_alpha	3.47
reg_lambda	4.91
min_child_samples	42